



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Airline Reservation Management System Using C

1. PSEUDOCODE

Pseudocode: Airline Reservation Management System

START

1. Open the airline data file.
2. If the file exists:
 Load passenger, flight and reservation records.
Otherwise:
 Start with empty records.
3. Display the Main Menu.
4. Repeat until the user selects Exit:

Display:

1. Add Passenger
2. Add Flight
3. Display Passengers
4. Display Flights
5. Make Reservation
6. Display Reservations
7. Search Passenger
8. Search Reservation

9. Update Passenger
10. Cancel Reservation
11. Sort Reservations by Priority
12. Display Seat Availability
13. Save Data
14. Exit

Read user choice.

If choice = 1:

- Create a passenger record.
- Store passenger details using structure.

If choice = 2:

- Create a flight record.
- Initialize all seats as available.

If choice = 3:

- Display all passenger records.

If choice = 4:

- Display available flight information.

If choice = 5:

- Read passenger ID and flight ID.
- Search for the passenger and flight.
- Check seat availability.
- Select an available seat.
- Read booking priority.

Calculate ticket fare.

Read payment method.

Store reservation details.

Mark selected seat as booked.

If choice = 6:

Display all reservation records.

If choice = 7:

Read passenger ID.

Search passenger using linear search.

Display passenger information.

If choice = 8:

Read reservation ID.

Search reservation.

Display reservation information.

If choice = 9:

Read passenger ID.

Update passenger details.

If choice = 10:

Read reservation ID.

Search reservation.

Change reservation status to Cancelled.

Release the booked seat.

If choice = 11:

Sort reservations according to booking priority
using Bubble Sort.

Display priority-wise reservations.

If choice = 12:

Read flight ID.

Display the seat availability.

If choice = 13:

Open data file in binary write mode.

Store passenger records.

Store flight records.

Store reservation records.

Close the file.

If choice = 14:

Save all records.

Display exit message.

STOP.

Otherwise:

Display "Invalid choice".

5. End program.

STOP

2. IMPLEMENTATION & RESULTS

2.1 Implementation

The Airline Reservation Management System was developed using the **C programming language**. The program is menu-driven and allows the user to manage passengers, flights, seats, reservations and payments.

The following C programming concepts were used in the implementation:

C Concept	How it is used
Structures	Store passenger, flight, reservation and payment details
Nested Structures	Address inside Passenger and Date inside Flight
Arrays	Store multiple passengers, flights and reservations
2D Array	Represent 5×10 aircraft seats
Functions	Perform separate operations like booking, searching and cancelling
Pointers	Access and modify structure records
Loops	Display records and process seats
If-Else	Check conditions such as seat availability
Switch	Handle the main menu
Linear Search	Search passenger, flight and reservation
Bubble Sort	Sort reservations by priority
File Handling	Save and load records using a file

2.2 Structures

Structures are used to store different types of information.

The main structures are:

Passenger

Flight

Reservation

Payment

Address

Date

2.3 Seat Management

A two-dimensional array is used for the aircraft seats:

```
int seats [5][10];
```

It represents **5 rows and 10 seats**, giving a total of **50 seats**.

[] → Available

[X] → Booked

ROW 1		[	]	[	]	[	]	[	]	[	]	[	]	[	]	[	]	[	]
ROW 2		[	]	[	X]	[	]	[	]	[	]	[	]	[	]	[	]	[	]
ROW 3		[	]	[	]	[	X]	[	]	[	]	[	]	[	]	[	]	[	]

2.4 Functions and Pointers

Different functions are used for different operations.

addPassenger()

addFlight()

makeReservation()

searchPassenger()

searchReservation()

cancelReservation()

updatePassenger()

sortReservationsByPriority()

saveData()

loadData()

2.5 Searching and Sorting

Linear Search is used to find passengers, flights and reservations using their IDs.

Passenger ID → Search → Display Record

Bubble Sort is used to arrange reservations according to priority.

1 → Emergency

2 → Business

3 → Normal

2.6 File Handling

File handling is used to permanently store the system data.

The file used is:

airline_data.dat

The Program Uses:

fopen()

fwrite()

fread()

fclose()

2.7 Results

The program was successfully compiled and executed.

The following operations were tested successfully:

1. Passenger registration
2. Flight creation
3. Displaying passengers
4. Displaying flights
5. Seat availability
6. Making reservations

7. Searching passengers
8. Searching reservations
9. Updating passenger details
10. Cancelling reservations
11. Sorting reservations by priority
12. Saving data
13. Loading previously saved data

Sample Seat Output

1. Flight creation

```
=====
                        ADD NEW FLIGHT
=====
Flight Number : AI202
Source       : Chennai
Destination  : Delhi
Date (DD MM YYYY): 15 09 2026
Departure Time: 09:30
Base Fare   : 5500

-----
Flight created successfully!
Flight ID    : F001
Total Seats  : 50
Available   : 50
-----
```

2. Actual aircraft seat map

```
=====
                        AIRCRAFT SEAT MAP
=====
Flight : AI202
Route  : Chennai -> Delhi

Legend: [ ] Available  [X] Booked

      WINDOW      AISLE      WINDOW

      1  2  3  4  5  6  7  8  9 10
      -----
ROW 1 | [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
ROW 2 | [ ] [ ] [ ] [X] [ ] [ ] [ ] [ ] [ ] [ ]
ROW 3 | [ ] [ ] [X] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
ROW 4 | [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
ROW 5 | [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]

-----
Available Seats : 47
Booked Seats    : 3
Total Seats     : 50
-----
```


3. Booking confirmation

```
=====
                        BOOKING CONFIRMED
=====
Reservation ID : R0004
Passenger      : Anand
Flight         : AI202
Route          : Chennai -> Delhi
Travel Date    : 15-09-2026
Departure      : 09:30
Seat           : Row 2 - Seat 4
Priority        : BUSINESS
Payment        : UPI
Ticket Amount  : Rs. 6352.50
Status         : Confirmed
=====
```

4. Cancellation

```
=====
                        RESERVATION CANCELLED
=====
Reservation : R0004
Released Seat: Row 2 - Seat 4
Status      : Cancelled
=====
```

5. File Storage Result

```
=====
DATA SAVED SUCCESSFULLY
File: airline_data.dat
Passengers : 2
Flights    : 1
Reservations : 3
=====
```

6. File Handling Output

```
=====
DATA SAVED SUCCESSFULLY
=====

File: airline_data.dat
Passengers   : 2
Flights      : 1
Reservations : 2
=====
```

4. TEST CASES

Test Case	Operation	Expected Result	Status
TC01	Add Passenger	Passenger ID generated	PASS
TC02	Add Flight	Flight created with 50 seats	PASS
TC03	Make Normal Reservation	Seat booked successfully	PASS
TC04	Make Emergency Reservation	Higher priority recorded	PASS
TC05	Search Passenger	Correct passenger displayed	PASS
TC06	Search Reservation	Correct reservation displayed	PASS
TC07	Update Passenger	Passenger information updated	PASS
TC08	Sort Reservations	Priority order displayed	PASS
TC09	Cancel Reservation	Reservation cancelled and seat released	PASS
TC10	Display Seat Availability	Available/booked seats displayed	PASS
TC11	Save Data	Records stored in file	PASS
TC12	Restart Program	Previous records loaded	PASS

5. DATA STRUCTURE / ALGORITHM COMPARISON

Alternative 1 – Arrays + Linear Search + Bubble Sort

The implemented system uses arrays of structures with Linear Search and Bubble Sort.

Advantages

- Easy to understand
- Simple implementation
- Direct access to records
- Easy to demonstrate in C
- Suitable for a small reservation system
- Requires less programming complexity

Disadvantages

- Fixed maximum size
- Linear search becomes slower as records increase
- Bubble Sort has $O(n^2)$ complexity

Alternative 2 – Linked List + Priority Queue

A linked-list-based system could dynamically store reservation records.

A priority queue could also be used to manage emergency and business bookings.

Advantages

- Dynamic memory allocation
- No fixed array size
- Efficient priority management
- Better for larger datasets

Disadvantages

- More difficult to implement
- Requires more pointer manipulation
- More complex debugging
- Higher implementation complexity for a beginner-level C project

Final Design Selection

The Array of Structures + Linear Search + Bubble Sort approach was selected because the proposed system is an academic airline reservation system with a limited number of records.

The design is easy to understand, easy to test and effectively demonstrates the required C programming concepts.

For a real-world airline system with millions of users, more advanced structures such as hash tables, balanced trees, priority queues and database systems would be more appropriate.

6. COMPLEXITY ANALYSIS

Operation	Algorithm	Time Complexity
Add Passenger	Array insertion	$O(1)$
Add Flight	Array insertion	$O(1)$
Search Passenger	Linear Search	$O(n)$
Search Reservation	Linear Search	$O(n)$
Search Flight	Linear Search	$O(n)$
Update Passenger	Linear Search	$O(n)$
Cancel Reservation	Linear Search	$O(n)$
Seat Availability	2D Array Traversal	$O(50)$, effectively $O(1)$
Priority Sorting	Bubble Sort	$O(n^2)$
Save Data	File Write	$O(n)$
Load Data	File Read	$O(n)$

Individual Contribution

Airline Reservation Management System Using C

I contributed to the design and implementation of the Airline Reservation Management System developed using the C programming language. My main contribution was to understand the problem requirements and convert them into a functional menu-driven C application. I worked on the design of the data structures required for the system. I implemented structures for Passenger, Flight, Reservation, Payment, Address and Date information. I also used nested structures, such as the Address structure inside the Passenger structure and the Date structure inside the Flight structure, to organize related information effectively. I contributed to the implementation of the passenger management module. This includes adding passenger records, displaying passenger information, searching passengers using Passenger ID and updating passenger details. Arrays of structures were used to store multiple passenger records. I also contributed to the flight and seat management module. The flight structure stores flight number, source, destination, travel date, departure time and base fare. A two-dimensional array was implemented to represent the aircraft seating arrangement. The system uses 5 rows and 10 seats per row, giving a total of 50 seats. The seat map displays [] for available seats and [X] for booked seats. Another important contribution was the reservation module. I implemented the process of selecting a passenger and flight, checking seat availability, selecting a seat, assigning booking priority, calculating the ticket fare and storing payment information. Emergency, Business and Normal priorities were implemented to support priority-based booking. I also contributed to searching and sorting operations. Linear Search was used to locate passenger, flight and reservation records. Bubble Sort was implemented to arrange reservations according to their priority. Emergency reservations are given higher priority than Business and Normal reservations. Pointers were also used in the implementation. Structure pointers were used to access and modify passenger and flight records efficiently. Functions were created for different operations to make the program modular and easier to maintain. I worked on the file-handling functionality of the system. The fopen(), fwrite(), fread() and fclose() functions were used to store and retrieve passenger, flight and reservation records in the binary file airline_data.dat. This allows the data to remain available even after the program is closed and restarted. Through this assignment, I gained practical knowledge of structures, nested structures, multidimensional arrays, pointers, functions, searching, sorting and file handling in C. I also learned how individual programming concepts can be combined to develop a complete real-world-oriented application

Conclusion

The Airline Reservation Management System was successfully implemented using the C programming language. The project demonstrates the practical use of structures, nested structures, arrays, multidimensional arrays, functions, pointers, searching, sorting, decision-making, loops and file handling.

The system provides essential airline reservation operations including passenger registration, flight management, seat allocation, reservation processing, payment management, priority-based booking, searching, updating and cancellation.

The use of a two-dimensional seat array provides a simple visual representation of aircraft seats, while priority-based sorting allows urgent bookings to be handled before normal bookings. Binary file handling provides persistent storage by saving and loading passenger, flight and reservation records.

The final implementation successfully satisfies the major requirements of the problem statement and provides a practical demonstration of C programming concepts in a real-world airline reservation scenario.

.

.